

Cahier d'Analyse et de Conception

Plateforme YowPainter Backend

API SaaS Multi-Tenant — Artistes & Collectionneurs

Document de référence technique

Architecture hexagonale · Multi-tenancy · Intégration Kernel · CamPay

Version logicielle	1.0.0-SNAPSHOT
Version document	1.0 — Juillet 2026
Stack	Spring Boot 4 · PostgreSQL 18 · Java 17
Classification	Documentation interne

Table des matières

1	Introduction	1
1.1	Objet du document	1
1.2	Présentation du projet	1
1.3	Périmètre	1
1.4	Stack technique	1
1.5	Conventions documentaires	1
1.6	Références	2
2	Contexte et environnement	3
2.1	Diagramme de contexte (niveau 0 DFD)	3
2.2	Acteurs du système	4
2.3	Modèle multi-tenant	4
3	Analyse des besoins	7
3.1	Besoins fonctionnels	7
3.2	Besoins non fonctionnels	7
3.3	Contraintes	7
3.4	Hypothèses	8
4	Cas d'utilisation	9
4.1	Diagramme général des cas d'utilisation	9
4.2	Description détaillée des cas d'utilisation principaux	9
4.2.1	UC-01 : Inscription Artiste	9
4.2.2	UC-02 : Validation Artiste par Admin	9
4.2.3	UC-03 : Achat d'une œuvre (Checkout)	9
4.2.4	UC-04 : Chat temps réel	9
4.3	Matrice acteur / cas d'utilisation	9
5	Architecture logicielle	13
5.1	Architecture hexagonale (Ports & Adapters)	13
5.2	Diagramme de packages	13
5.3	Structure interne d'un module type	13
5.4	Diagramme de composants	14
5.5	Modules fonctionnels	14
5.6	Couche Kernel partagée	14
6	Modèle de classes métier	17
6.1	Diagramme de classes — Noyau utilisateurs	17
6.2	Diagramme de classes — Artworks	17
6.3	Diagramme de classes — Commerce	17
6.4	Diagramme de classes — Événements et Chat	17
6.5	Modèle entité-relation global	17

6.6	Statuts métier clés	17
7	Modèle de classes technique	23
7.1	Diagramme de classes — Couche Auth (exemple module)	23
7.2	Diagramme de classes — Intégration Kernel	25
7.3	Diagramme de classes — Multi-tenancy	25
7.4	Diagramme de classes — Paiement CamPay	25
7.5	Diagramme de classes — Sécurité	25
7.6	Inventaire des ports de sortie (Repository)	25
8	Diagrammes de séquence	27
8.1	Séquence — Authentification (Login)	27
8.2	Séquence — Provisioning artiste (validation admin)	27
8.3	Séquence — Checkout boutique (CamPay)	27
8.4	Séquence — Chat WebSocket STOMP	27
8.5	Séquence — Sync Kafka (optionnel)	27
8.6	Machine d'états — Cycle de vie artiste	27
9	Déploiement et infrastructure	31
9.1	Diagramme de déploiement — Environnement local	31
9.2	Diagramme de déploiement — Production (cible)	31
9.3	Variables d'environnement critiques	31
9.4	Pipeline de build et déploiement	32
9.5	Monitoring et observabilité	32
10	Référence API REST	35
10.1	Conventions générales	35
10.2	Matrice des endpoints par module	35
10.2.1	Authentification — /api/auth	35
10.2.2	Profil — /api/me	35
10.2.3	Artiste — /api	35
10.2.4	Œuvres — /api	36
10.2.5	Boutique — /api/shop	36
10.2.6	Événements — /api	36
10.2.7	Paiement et Wallet	37
10.2.8	Administration — /api/admin	37
10.3	WebSocket STOMP	37
10.4	Codes de réponse HTTP	37
11	Annexes	39
11.1	Glossaire	39
11.2	Matrice de traçabilité besoins / modules	40
11.3	Structure des schémas de base de données	40
11.3.1	Schéma public	40
11.3.2	Schéma tenant_{organizationId}	40
11.4	Checklist de mise en production	40
11.5	Historique des versions du document	41
11.6	Index des figures	41

Table des figures

2.1	Diagramme de contexte — écosystème YowPainter Backend	3
2.2	Isolation des données — schema-per-tenant	5
4.1	Diagramme de cas d'utilisation — vue globale	10
5.1	Architecture hexagonale — découpage en couches	13
5.2	Diagramme de packages — structure modulaire	14
5.3	Diagramme de composants — flux de dépendances	14
6.1	Classes métier — Utilisateurs, wallet et abonnements	18
6.2	Classes métier — Module Artwork	19
6.3	Classes métier — Module Shop / Payment	20
6.4	Classes métier — Événements et Chat	21
6.5	Modèle entité-relation simplifié	22
7.1	Classes techniques — Module Auth (pattern hexagonal)	23
7.2	Classes techniques — Couche Kernel partagée	24
7.3	Classes techniques — Multi-tenancy Hibernate	24
7.4	Classes techniques — Module Payment	25
7.5	Classes techniques — Sécurité JWT	26
8.1	Séquence — Connexion utilisateur	27
8.2	Séquence — Provisioning artiste après approbation admin	28
8.3	Séquence — Paiement commande boutique via CamPay	28
8.4	Séquence — Envoi message chat temps réel	29
8.5	Séquence — Synchronisation statut commande via Kafka	29
8.6	Machine d'états — Statut artiste	30
9.1	Diagramme de déploiement — Développement local	31
9.2	Diagramme de déploiement — Architecture production cible	33

Liste des tableaux

1.1	Périmètre fonctionnel du backend	1
1.2	Technologies utilisées	2
2.1	Acteurs et responsabilités	4
3.1	Catalogue des besoins fonctionnels	7
3.2	Exigences non fonctionnelles	8
4.1	Matrice de traçabilité acteurs cas d'utilisation	11
5.1	Inventaire des 14 modules métier	15
5.2	Ports Kernel (shared/kernel/port)	15
6.1	Énumérations et statuts métier	17
7.1	Ports de persistance par module	25
9.1	Variables d'environnement principales	31
9.2	Points de contrôle disponibles	32
10.1	Endpoints Auth	35
10.2	Endpoints Artiste	36
10.3	Endpoints Artwork	36
10.4	Endpoints Shop	36
10.5	Endpoints Event	36
10.6	Endpoints Payment / Wallet	37
10.7	Endpoints Admin	37
10.8	Configuration WebSocket	37
10.9	Codes HTTP standards utilisés	38
11.1	Glossaire des termes	39
11.2	Traçabilité BF → Modules	40
11.3	Tables du schéma public	40
11.4	Tables par tenant artiste	41
11.5	Historique documentaire	41

Chapitre 1

Introduction

1.1 Objet du document

Ce **cahier d’analyse et de conception (CAC)** constitue la *documentation de référence* du backend YowPainter. Il décrit l’ensemble du système : contexte métier, acteurs, cas d’utilisation, architecture logicielle, modèles de données, intégrations externes, sécurité et déploiement.

Ce document s’adresse aux développeurs, architectes, testeurs et parties prenantes techniques souhaitant comprendre, maintenir ou faire évoluer la plateforme.

1.2 Présentation du projet

YowPainter est une plateforme **SaaS multi-tenant** dédiée aux **artistes visuels** et aux **collectionneurs**. Elle permet :

- aux artistes de créer un espace personnel (tenant isolé), publier des œuvres, gérer une boutique en ligne, organiser des événements et recevoir des paiements ;
- aux acheteurs de découvrir des artistes, acheter des œuvres, réserver des billets et interagir (likes, commentaires, chat) ;
- aux administrateurs de valider les inscriptions artistes, superviser les tenants et consulter les statistiques globales.

1.3 Périmètre

TABLE 1.1 – Périmètre fonctionnel du backend

Inclus	Authentification JWT, multi-tenancy, œuvres, boutique, événements, paiements CamPay, wallet, abonnements, chat Web-Socket, notifications, recherche, administration
Exclus	Frontend Next.js, interface utilisateur Kernel RT-Comops, infrastructure réseau (load balancers, CDN)

1.4 Stack technique

1.5 Conventions documentaires

- Les diagrammes UML sont réalisés en pour une compilation LaTeX autonome.
- Les chemins de packages suivent la convention `com.yowpainter.*`.

TABLE 1.2 – Technologies utilisées

Composant	Technologie / Version
Langage	Java 17
Framework	Spring Boot 4.0.0 (Spring Framework 7.0)
Sécurité	Spring Security 7.0, OAuth2 Resource Server, JWT RS256
Persistance	Spring Data JPA, Hibernate 6, PostgreSQL 18
Multi-tenancy	Schema-per-tenant (Hibernate)
Migrations	Flyway 11.0.0
Paielements	CamPay (Mobile Money MTN, Orange, MoMo Pay)
Messagerie	WebSocket STOMP (Spring)
Événements	Apache Kafka (optionnel, sync Kernel)
Documentation API	OpenAPI 3 / SpringDoc 3.0.0
E-mail	Spring Boot Mail
Build	Maven 3.9+

- Les endpoints API sont préfixés par `/api`.
- Les identifiants UUID sont notés `UUID` sauf mention contraire.

1.6 Références

- `README.md` — Vue d'ensemble et installation
- `docs/KERNEL_INTEGRATION.md` — Intégration Kernel RT-Comops
- `docs/ARTIST_APPROVAL_WORKFLOW.md` — Workflow validation artiste
- `docs/CONFIGURATION_KERNEL_YOWPAINTER.md` — Configuration locale
- Swagger UI : <http://localhost:8090/swagger-ui.html>

Chapitre 2

Contexte et environnement

2.1 Diagramme de contexte (niveau 0 DFD)

La figure 2.1 situe le système YowPainter Backend au centre de son écosystème. Le frontend ne communique **jamais** directement avec le Kernel ; seul le backend YowPainter est une *ClientApplication* autorisée.

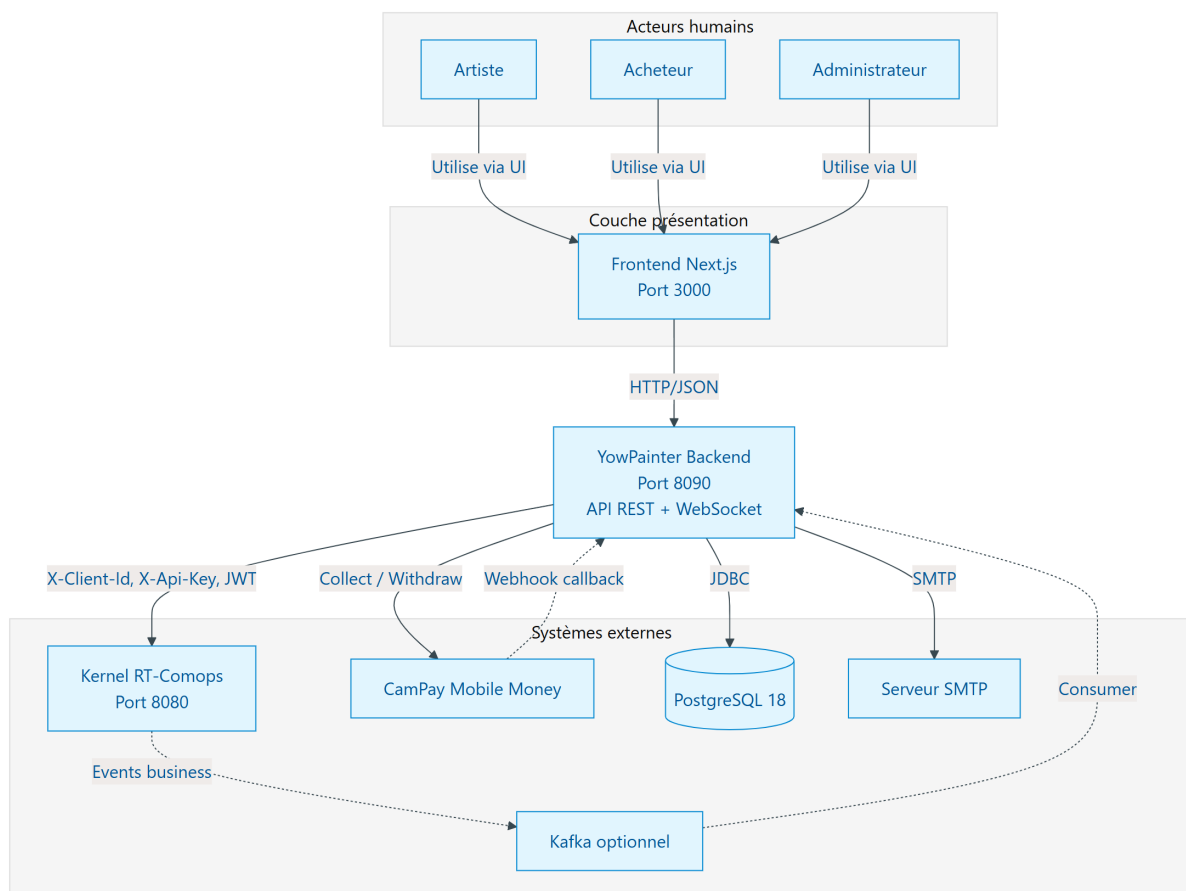


FIGURE 2.1 – Diagramme de contexte — écosystème YowPainter Backend

TABLE 2.1 – Acteurs et responsabilités

Acteur	Rôle applicatif	Responsabilités principales
Artiste	ROLE_ARTIST	Gérer profil, œuvres, boutique, événements, wallet, abonnement
Acheteur	ROLE_BUYER	Acheter, réserver billets, liker, commenter, chatter
Administrateur	ROLE_ADMIN	Valider artistes, gérer tenants, statistiques, logs
Frontend	Système client	Consommer l'API REST/WebSocket (JWT utilisateur)
Kernel	RT- Système	Auth, org, produits, commandes, fichiers, notifications
Comops	IAM/Commerce	
CamPay	PSP Mobile Money	Collecte et retrait Mobile Money (XAF)
PostgreSQL	SGBD	Persistance multi-schémas
Kafka	Bus événements	Sync statuts commandes (optionnel)

2.2 Acteurs du système

2.3 Modèle multi-tenant

Chaque artiste actif possède un **schéma PostgreSQL dédié** (`tenant_{organizationId}`). Les données globales (utilisateurs, artistes, wallet, chat) résident dans le schéma **public**.

Résolution du tenant :

1. Header HTTP `X-Organization-Id` (contexte explicite)
2. Artiste authentifié → déduction via `organizationId`
3. Routes publiques `/v1/public/{artistSlug}/...` → résolution par slug

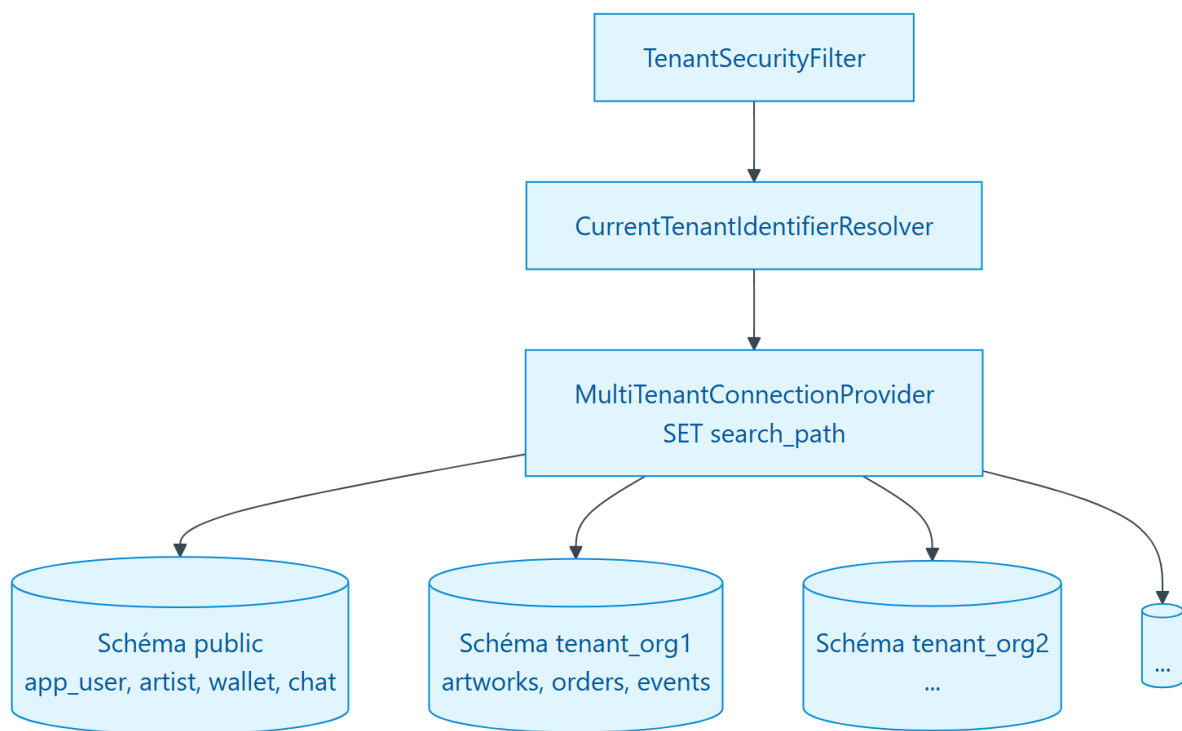


FIGURE 2.2 – Isolation des données — schema-per-tenant

Chapitre 3

Analyse des besoins

3.1 Besoins fonctionnels

TABLE 3.1: Catalogue des besoins fonctionnels

ID	Module	Description
BF-01	Auth	Inscription artiste/acheteur avec confirmation e-mail via Kernel
BF-02	Auth	Connexion, refresh token, déconnexion (proxy Kernel)
BF-03	Auth	Réinitialisation mot de passe par e-mail
BF-04	Admin	Validation manuelle des inscriptions artistes (provisioning org Kernel)
BF-05	Artist	Profil artiste avec slug unique et analytics
BF-06	Artwork	CRUD œuvres, images/vidéos, likes, commentaires
BF-07	Shop	Catalogue produits, commandes, checkout Mobile Money
BF-08	Event	Création événements, réservations, billets QR
BF-09	Payment	Intégration CamPay (collect, callback, polling)
BF-10	Payment	Wallet artiste, commissions, retraits
BF-11	Subscription	Plans FREE/PRO/ELITE avec taux de commission différenciés
BF-12	Chat	Messagerie instantanée REST + WebSocket STOMP
BF-13	Notification	Notifications utilisateur (source Kernel)
BF-14	News	Actualités artiste
BF-15	Search	Recherche globale multi-entités
BF-16	File	Upload fichiers délégué au Kernel
BF-17	Admin	Gestion tenants, utilisateurs, statistiques

3.2 Besoins non fonctionnels

3.3 Contraintes

- Le frontend **ne doit jamais** appeler le Kernel avec **X-Api-Key**.
- Les tokens JWT sont émis par le Kernel (RS256) ; YowPainter est un resource server.

TABLE 3.2 – Exigences non fonctionnelles

Catégorie	Exigence
Performance	API REST stateless, temps de réponse < 500ms pour requêtes simples
Scalabilité	Isolation par schéma permettant le sharding futur par tenant
Sécurité	JWT RS256 (JWKS Kernel), CORS configuré, routes protégées par rôle
Disponibilité	Health checks (/api/public/health, /api/system/health)
Maintenabilité	Architecture hexagonale modulaire, tests unitaires par service
Traçabilité	Logs structurés, provisioning artiste tracé ([provision])
Portabilité	Configuration externalisée (variables d'environnement)
Conformité	Pas de stockage direct des credentials CamPay côté client

- Le provisioning artiste en production requiert un code MFA bootstrap admin.
- Les notifications sont centralisées dans le Kernel (pas de double écriture locale).
- `ddl-auto: none` — toute évolution de schéma passe par Flyway.

3.4 Hypothèses

- Le Kernel RT-Comops est déployé et accessible avant le backend YowPainter.
- Une `ClientApplication yowpainter-backend` est enregistrée dans le Kernel.
- PostgreSQL 18 est disponible avec droits de création de schémas.
- CamPay est configuré avec credentials valides pour l'environnement cible.

Chapitre 4

Cas d'utilisation

4.1 Diagramme général des cas d'utilisation

La figure 4.1 présente les cas d'utilisation principaux regroupés par acteur.

4.2 Description détaillée des cas d'utilisation principaux

4.2.1 UC-01 : Inscription Artiste

Identifiant	UC-01
Nom	Inscription Artiste
Acteur principal	Artiste (non authentifié)
Préconditions	Kernel accessible, e-mail valide
Postconditions	Compte créé, statut <code>PENDING_EMAIL</code> puis <code>PENDING_APPROVAL</code>
Scénario nominal	1. L'artiste envoie <code>POST /api/auth/register</code> avec <code>ROLE_ARTIST</code>. 2. Le backend appelle Kernel sign-up (acteur <code>BUSINESS/ART</code>). 3. Création locale <code>AppUser</code> + <code>Artist</code> (slug auto-généré). 4. Retour avec message de confirmation e-mail. 5. Artiste confirme via <code>POST /api/auth/confirm-email</code>. 6. Statut passe à <code>PENDING_APPROVAL</code>.
Extensions	E1 : E-mail déjà utilisé → 409 Conflict.
E2 : Kernel indisponible → 500 avec message explicite.	

4.2.2 UC-02 : Validation Artiste par Admin

4.2.3 UC-03 : Achat d'une œuvre (Checkout)

4.2.4 UC-04 : Chat temps réel

4.3 Matrice acteur / cas d'utilisation

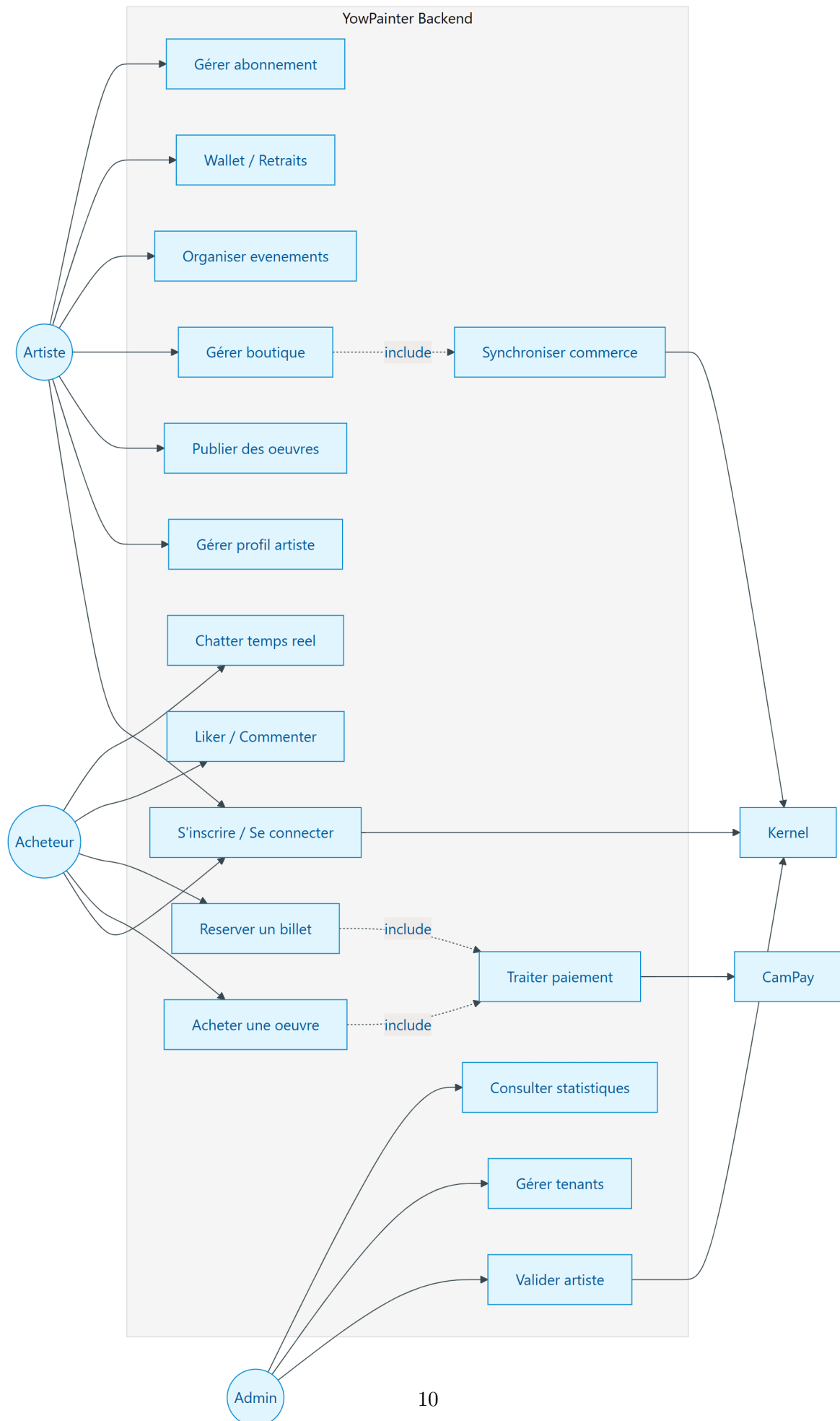


FIGURE 4.1. Diagramme de cas d'utilisation — vue globale

Identifiant	UC-02
Nom	Validation et provisioning artiste
Acteur principal	Administrateur (ROLE_ADMIN)
Préconditions	Artiste en PENDING_APPROVAL, credentials bootstrap valides
Postconditions	Artiste ACTIVE, org Kernel créée, schéma tenant migré
Scénario nominal	1. Admin liste GET /api/admin/artists/pending. 2. Admin approuve POST /api/admin/artists/{id}/approve + MFA. 3. KernelArtistProvisioningService : login bootstrap, création org, plan COMMERCE, rôle ORGANIZATION_ADMIN. 4. Migration Flyway du schéma tenant. 5. Statut artiste → ACTIVE.

Identifiant	UC-03
Nom	Acheter une œuvre via Mobile Money
Acteur principal	Acheteur (ROLE_BUYER)
Préconditions	Produit disponible, acheteur authentifié
Postconditions	Commande PAID, wallet artiste crédité (net commission)
Scénario nominal	1. Création commande POST /api/shop/v1/public/{slug}/orders. 2. Checkout POST /api/shop/orders/{id}/checkout?phoneNumber=.... 3. PaymentService → CamPay collect (USSD push). 4. Callback POST /api/payment/callback confirme le paiement. 5. Confirmation commande Kernel + crédit wallet + notification.

Identifiant	UC-04
Nom	Envoyer un message instantané
Acteurs	Artiste, Acheteur
Préconditions	Connexion WebSocket STOMP avec JWT valide
Scénario nominal	1. Client se connecte à /ws avec header Authorization: Bearer 2. Envoi message vers /app/chat. 3. ChatController.processMessage persiste ChatMessage. 4. Dispatch vers /user/queue/messages du destinataire.

TABLE 4.1 – Matrice de traçabilité acteurs cas d'utilisation

	Artiste	Acheteur	Admin	Système
Auth / Profil				Kernel
Gestion œuvres				
Boutique / Commandes				Kernel, CamPay
Événements / Billets				CamPay
Wallet / Retraits				CamPay
Abonnements				CamPay
Chat				
Validation artistes				Kernel
Administration				

Chapitre 5

Architecture logicielle

5.1 Architecture hexagonale (Ports & Adapters)

Le backend suit une **architecture hexagonale modulaire**. Chaque module métier (auth, artwork, shop, etc.) est découpé en trois couches :

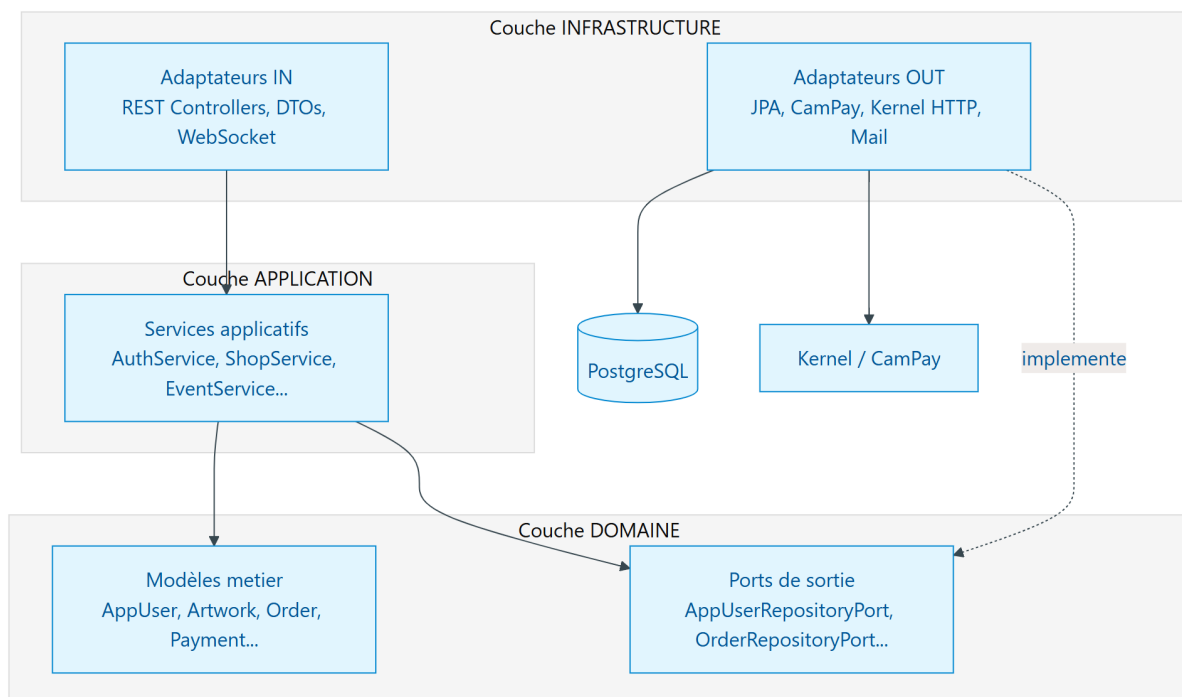


FIGURE 5.1 – Architecture hexagonale — découpage en couches

5.2 Diagramme de packages

La figure 5.2 illustre l'organisation modulaire du code source.

5.3 Structure interne d'un module type

Chaque module sous `com.yowpainter.modules.*` respecte la même arborescence :

```
com.yowpainter.modules.<module>/  
    domain/
```

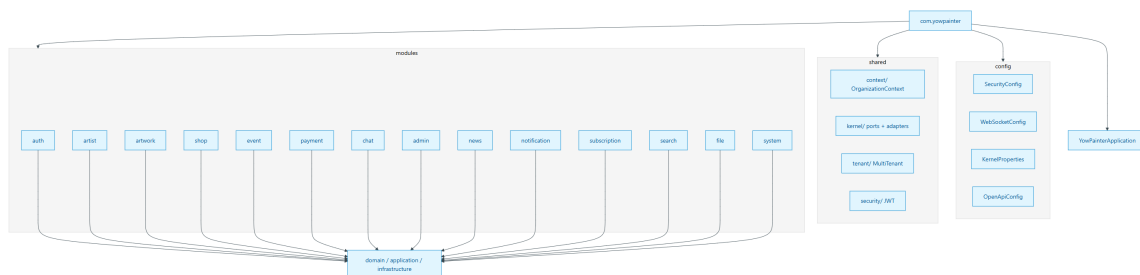


FIGURE 5.2 – Diagramme de packages — structure modulaire

```

model/           # Entites metier pures
port/out/        # Interfaces RepositoryPort
application/
service/         # Cas d'utilisation (orchestration)
infrastructure/
adapter/
in/web/          # Controllers REST + DTOs
out/             # JPA, clients HTTP, etc.

```

5.4 Diagramme de composants

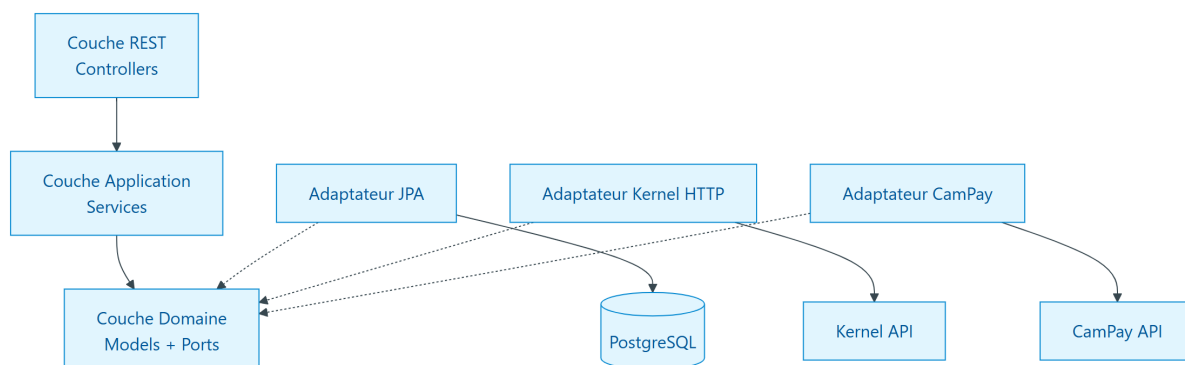


FIGURE 5.3 – Diagramme de composants — flux de dépendances

5.5 Modules fonctionnels

5.6 Couche Kernel partagée

Le package `com.yowpainter.shared.kernel` centralise l'intégration avec le Kernel RT-Comops. Il expose 7 ports hexagonaux :

TABLE 5.1 – Inventaire des 14 modules métier

Module	Service principal	Responsabilité
auth	AuthService	Authentification, profils, proxy Kernel auth
artist	ArtistService	Profil artiste, slug, analytics
artwork	ArtworkService	Œuvres, médias, likes, commentaires
shop	ShopService	Produits, commandes, commerce Kernel
event	EventService	Événements, réservations, billets QR
payment	PaymentService	CamPay, wallet, retraits, polling
subscription	SubscriptionService	Plans FREE/PRO/ELITE
chat	ChatMessageService	Messagerie REST + WebSocket
notification	NotificationService	Notifications via Kernel
news	NewsService	Actualités artiste
admin	AdminService	Administration plateforme
file	FileService	Upload délégué Kernel
search	— (agrégation)	Recherche globale multi-services
system	—	Health, version, metrics

TABLE 5.2 – Ports Kernel (shared/kernel/port)

Port	Responsabilité
KernelAuthPort	Sign-up, login, refresh, logout, reset password
KernelOrganizationPort	Création/approbation org, plans commerciaux
KernelAdministrationPort	Rôles, assignation utilisateurs
KernelActorPort	Onboarding acteurs BUSINESS/ART
KernelProductPort	CRUD produits commerce
KernelSalesPort	Commandes, confirm/cancel
KernelFilePort	Upload fichiers
KernelNotificationPort	Envoi/récupération notifications

Chapitre 6

Modèle de classes métier

Ce chapitre présente le **modèle conceptuel de domaine** : entités métier, énumérations et relations, indépendamment des détails techniques d'implémentation.

6.1 Diagramme de classes — Noyau utilisateurs

6.2 Diagramme de classes — Artworks

6.3 Diagramme de classes — Commerce

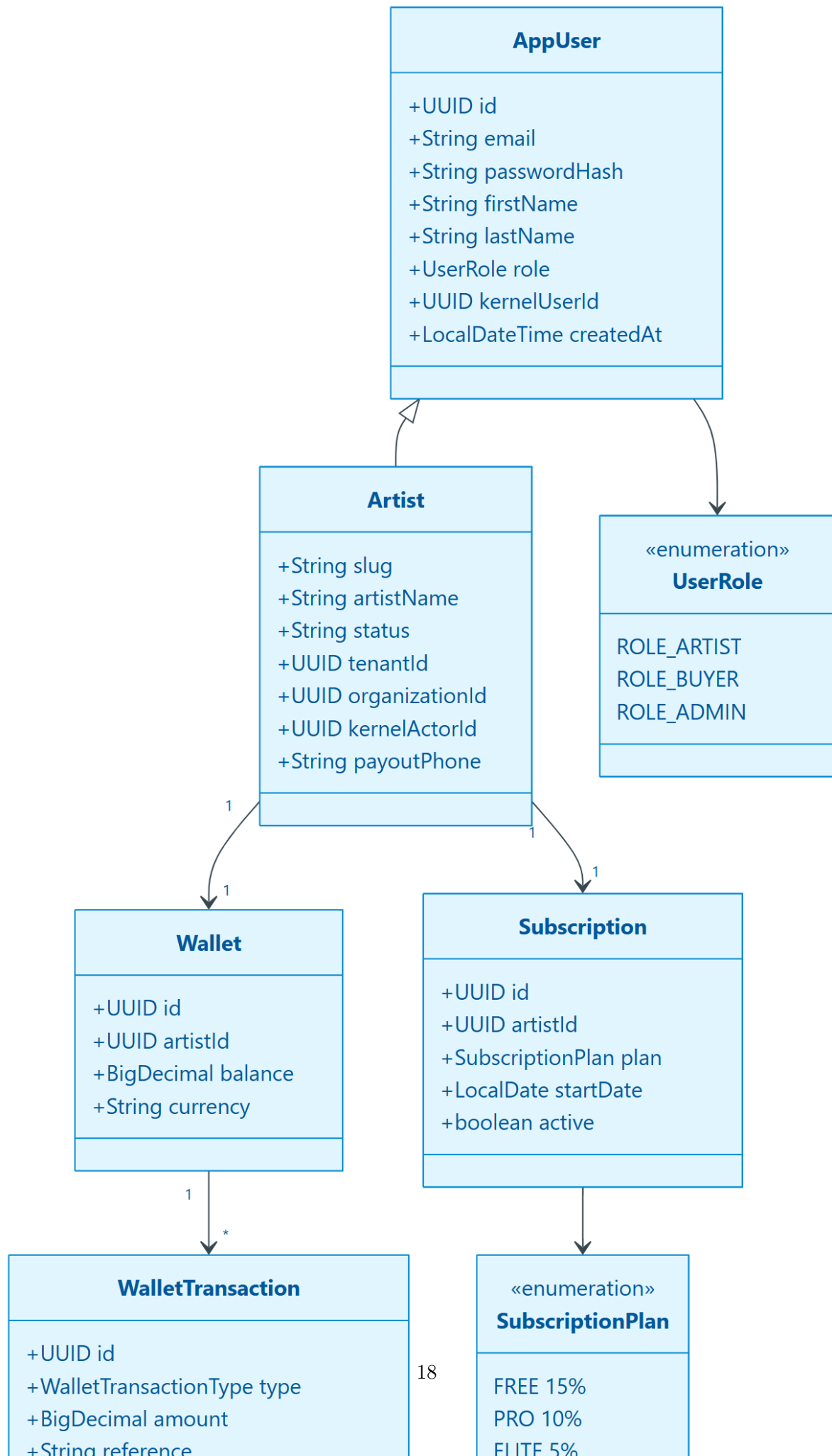
6.4 Diagramme de classes — Événements et Chat

6.5 Modèle entité-relation global

6.6 Statuts métier clés

TABLE 6.1 – Énumérations et statuts métier

Entité	Statuts / Valeurs	Signification
Artist.status	PENDING_EMAIL, PENDING_APPROVAL, ACTIVE, REJECTED	Cycle de vie ins
Artwork.status	DRAFT, PUBLISHED, ARCHIVED, SOLD	Visibilité et dis
Order.status	PENDING, PAID, SHIPPED, DELIVERED, CANCELLED	Cycle commande
Payment.status	PENDING, SUCCESS, FAILED, EXPIRED	État transaction
Reservation.status	PENDING, CONFIRMED, CANCELLED, EXPIRED	Réservation bil
SubscriptionPlan	FREE (15%), PRO (10%), ELITE (5%)	Taux commissio



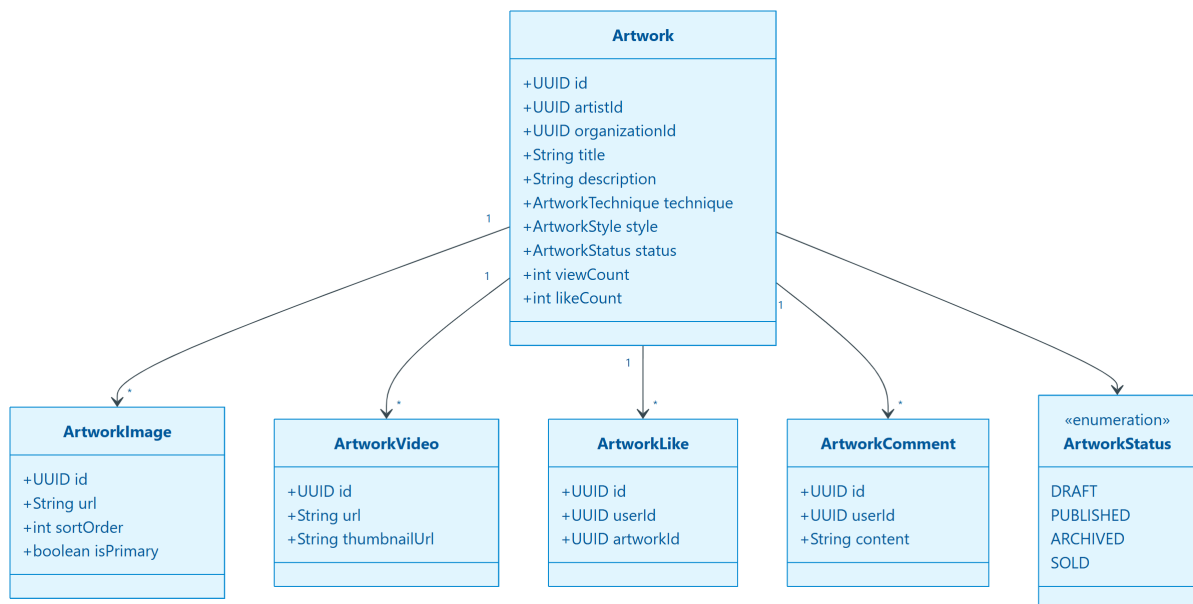


FIGURE 6.2 – Classes métier — Module Artwork

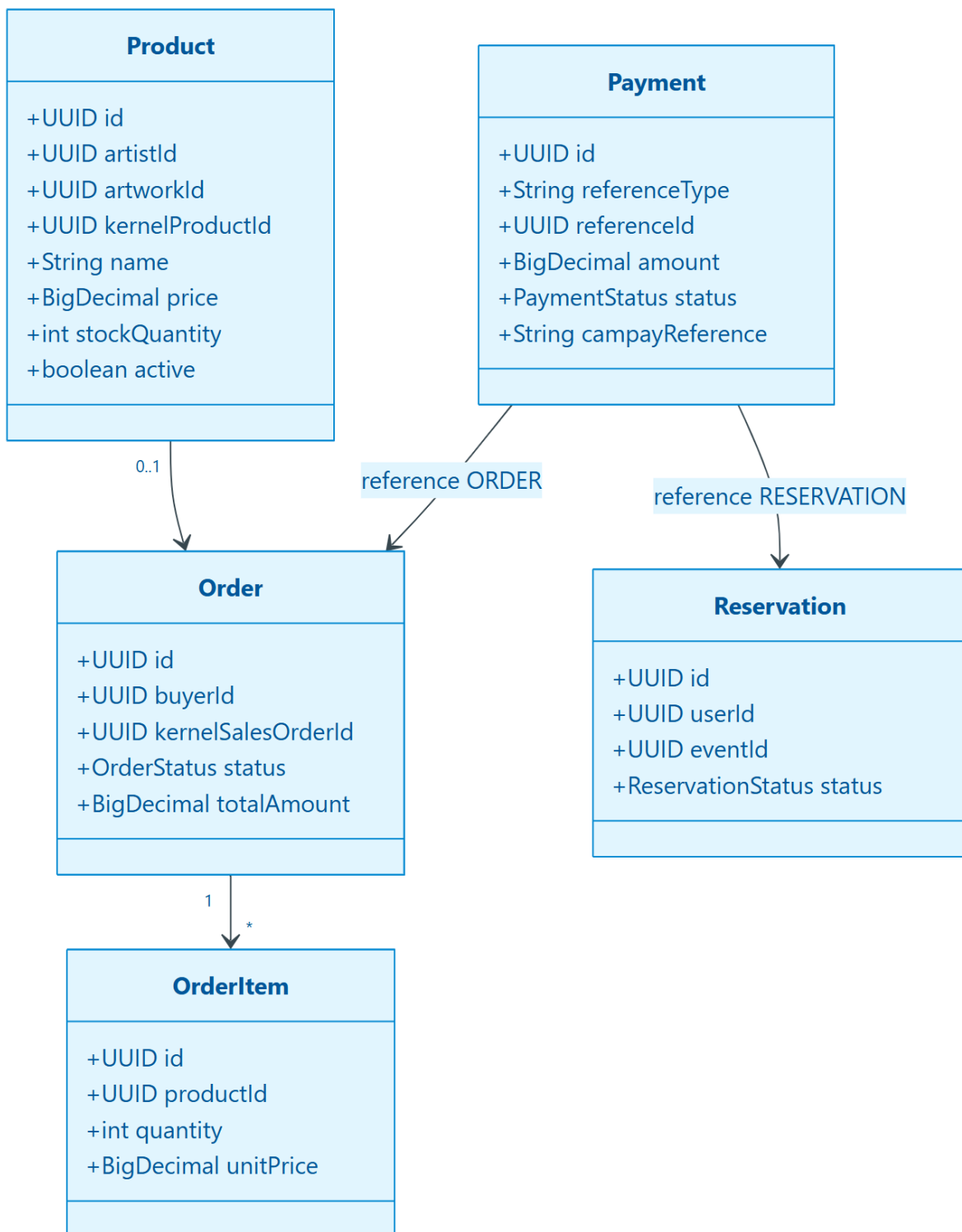


FIGURE 6.3 – Classes métier — Module Shop / Payment

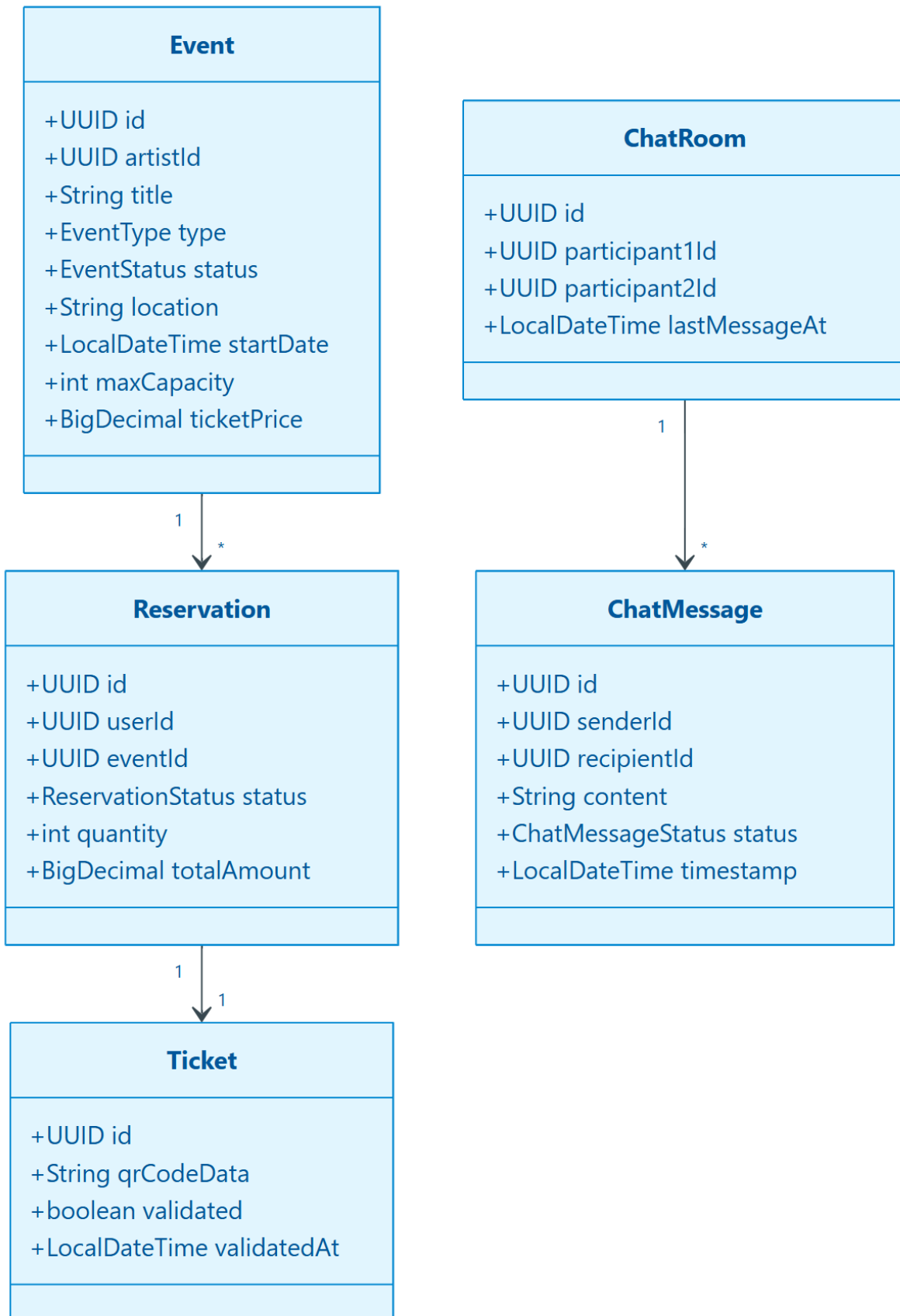


FIGURE 6.4 – Classes métier — Événements et Chat

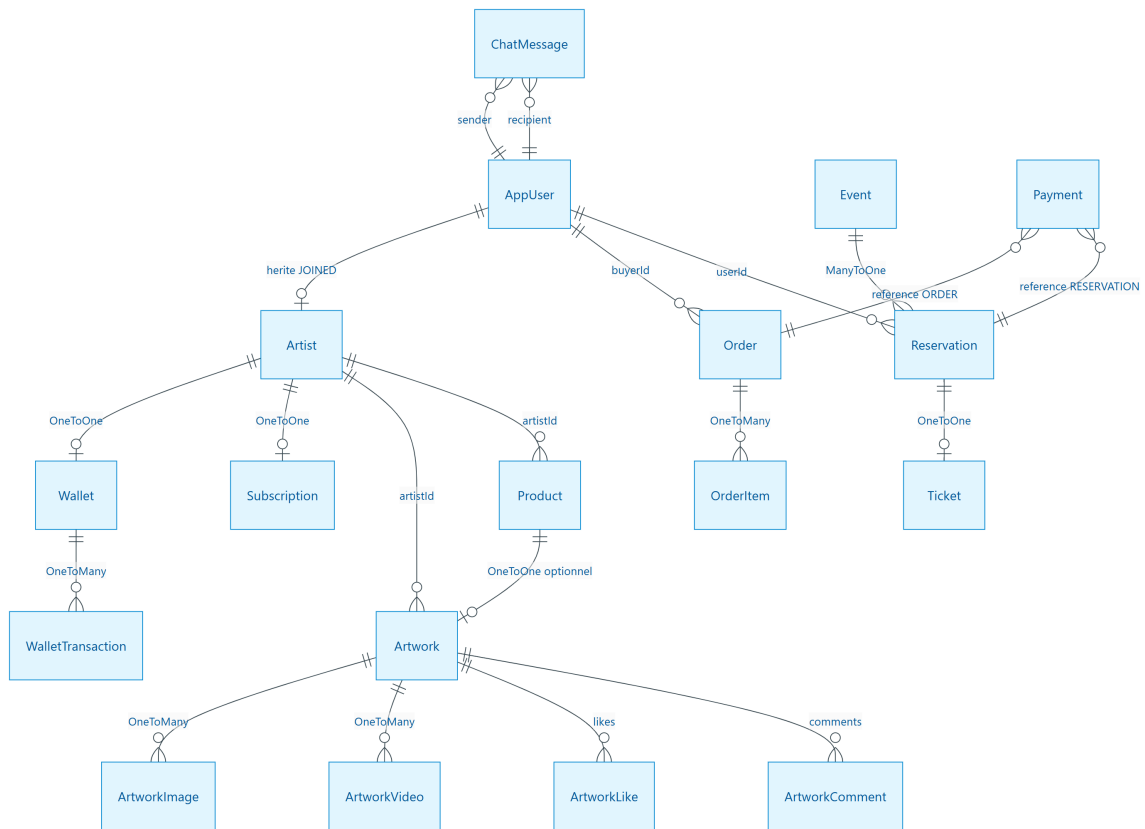


FIGURE 6.5 – Modèle entité-relation simplifié

Chapitre 7

Modèle de classes technique

Ce chapitre décrit les **classes d'infrastructure**, les ports, les adaptateurs et les composants transverses qui implémentent l'architecture hexagonale.

7.1 Diagramme de classes — Couche Auth (exemple module)

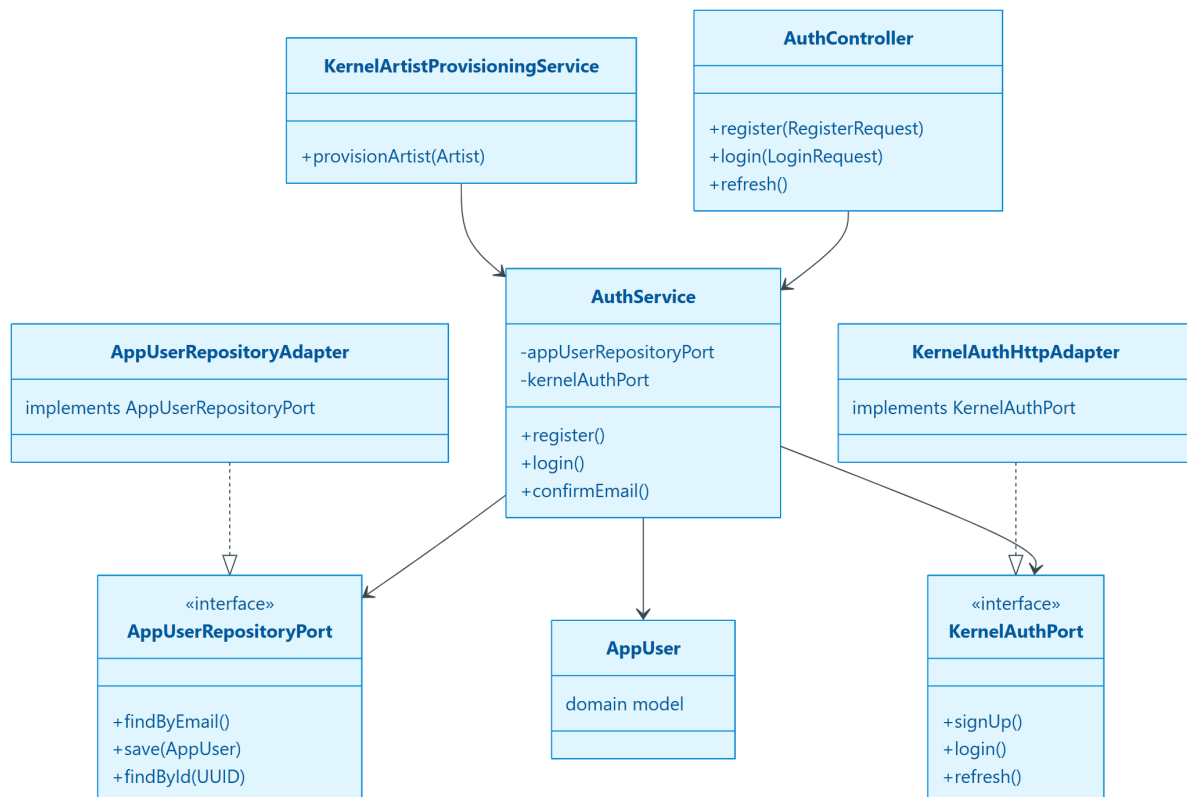


FIGURE 7.1 – Classes techniques — Module Auth (pattern hexagonal)

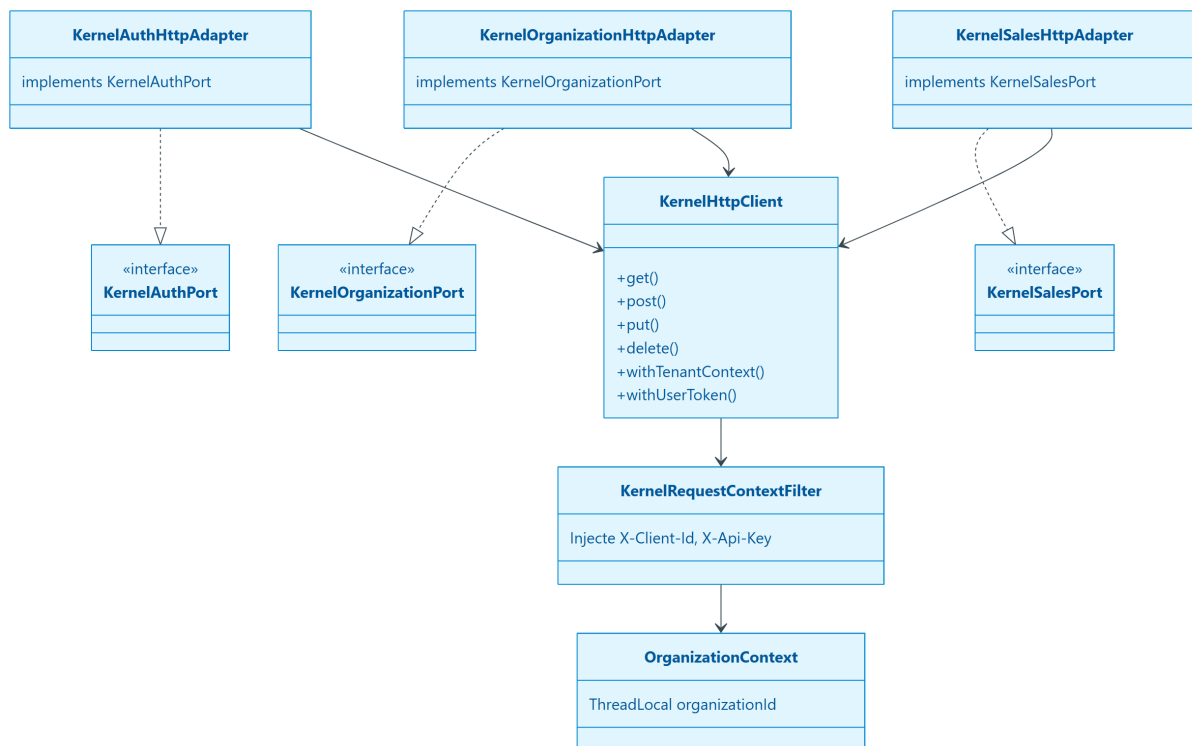


FIGURE 7.2 – Classes techniques — Couche Kernel partagée

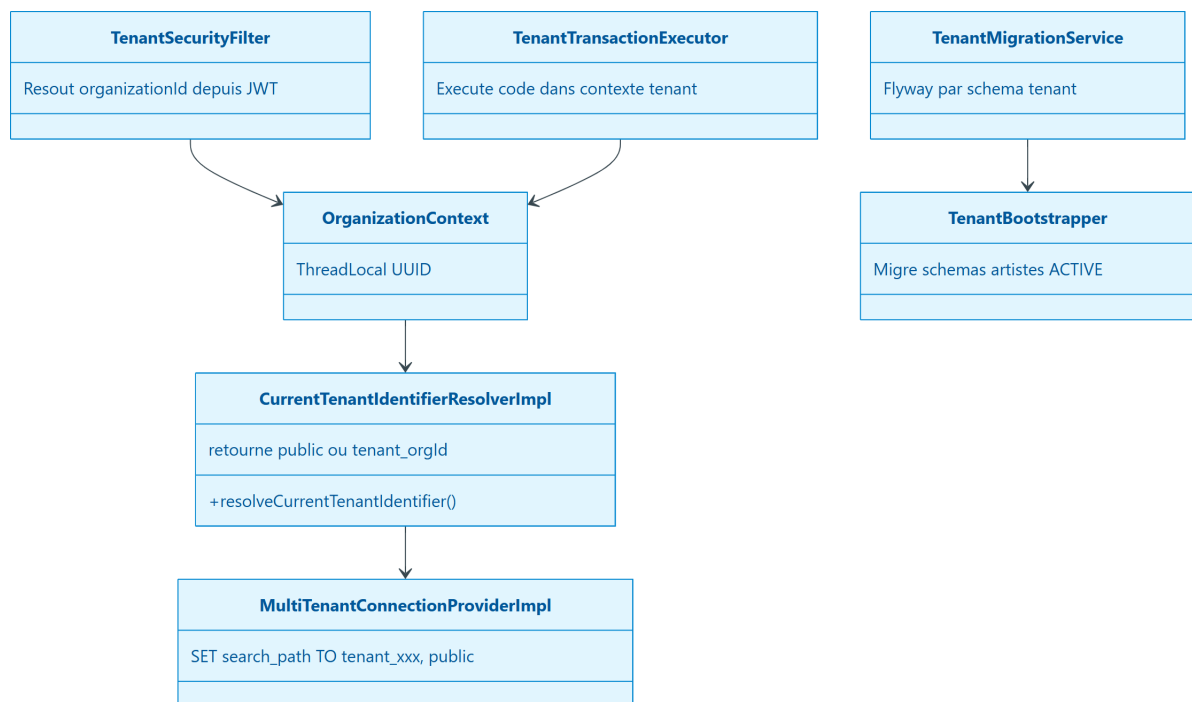


FIGURE 7.3 – Classes techniques — Multi-tenancy Hibernate

7.2 Diagramme de classes — Intégration Kernel

7.3 Diagramme de classes — Multi-tenancy

7.4 Diagramme de classes — Paiement CamPay

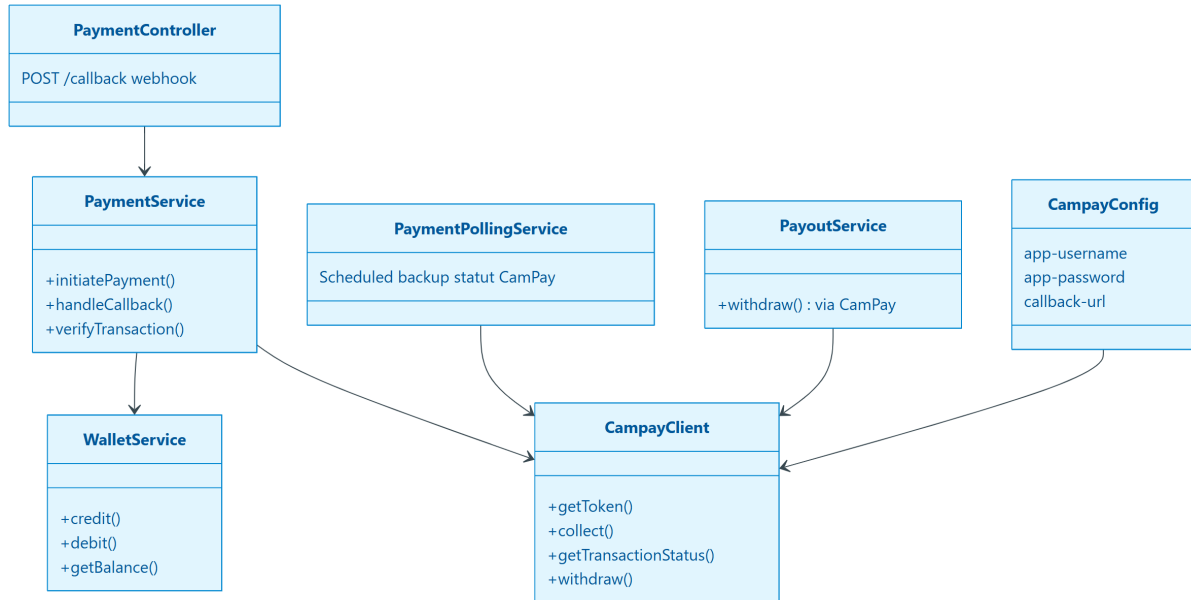


FIGURE 7.4 – Classes techniques — Module Payment

7.5 Diagramme de classes — Sécurité

7.6 Inventaire des ports de sortie (Repository)

TABLE 7.1: Ports de persistance par module

Module	Port(s)
auth	AppUserRepositoryPort, RefreshTokenRepositoryPort
artist	ArtistRepositoryPort, PendingProvisionSessionRepositoryPort
artwork	ArtworkRepositoryPort, ArtworkLikeRepositoryPort, ArtworkCommentRepositoryPort
shop	ProductRepositoryPort, OrderRepositoryPort, PaymentRepositoryPort
event	EventRepositoryPort, ReservationRepositoryPort, TicketRepositoryPort
payment	WalletRepositoryPort, WalletTransactionRepositoryPort
subscription	SubscriptionRepositoryPort
chat	ChatMessageRepositoryPort, ChatRoomRepositoryPort
news	NewsRepositoryPort

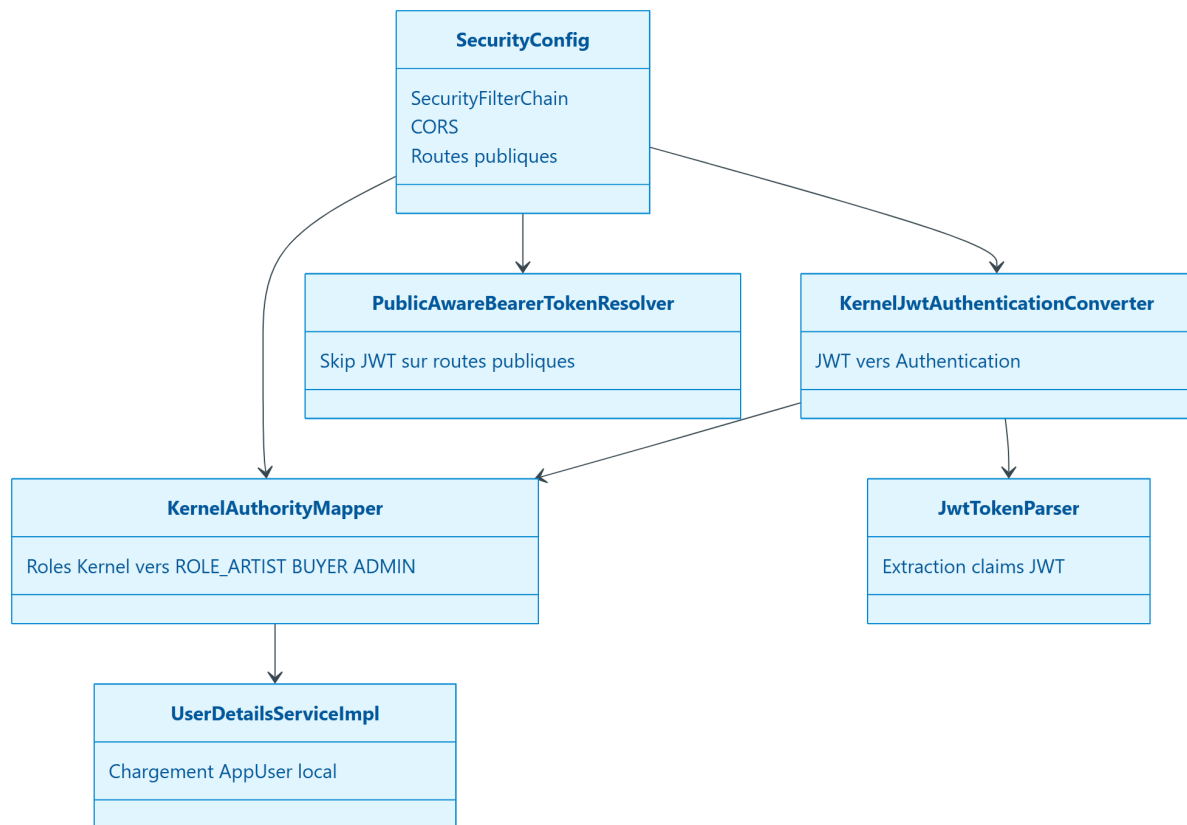


FIGURE 7.5 – Classes techniques — Sécurité JWT

Chapitre 8

Diagrammes de séquence

8.1 Séquence — Authentification (Login)

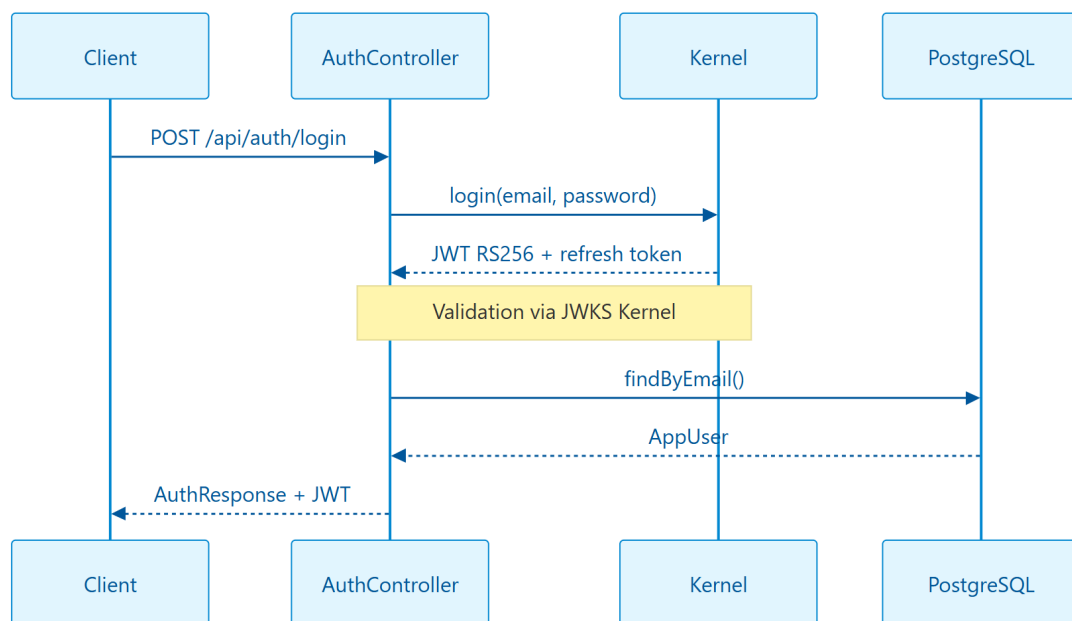


FIGURE 8.1 – Séquence — Connexion utilisateur

8.2 Séquence — Provisioning artiste (validation admin)

8.3 Séquence — Checkout boutique (CamPay)

8.4 Séquence — Chat WebSocket STOMP

8.5 Séquence — Sync Kafka (optionnel)

8.6 Machine d'états — Cycle de vie artiste

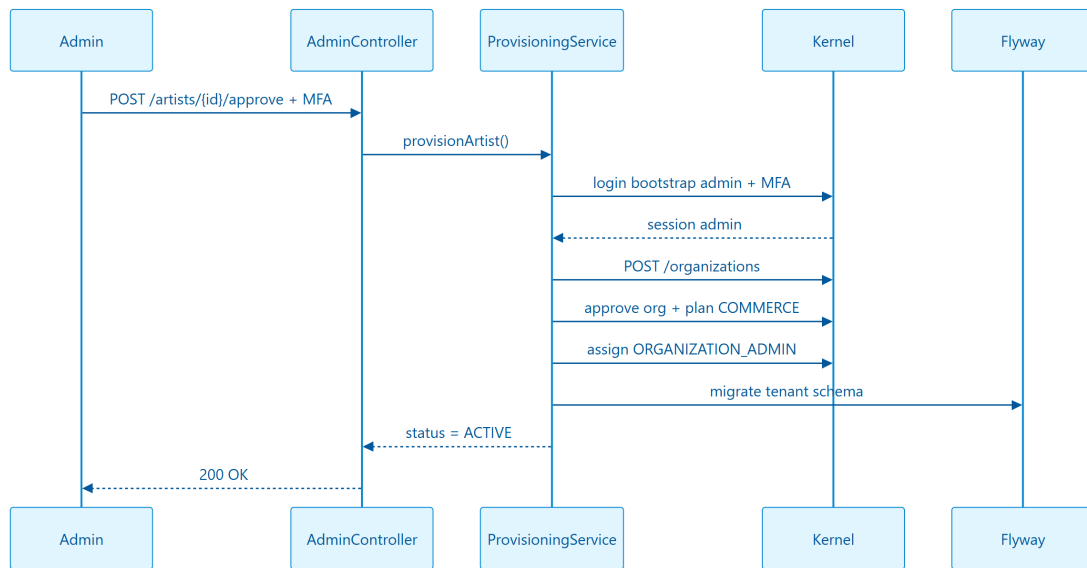


FIGURE 8.2 – Séquence — Provisioning artiste après approbation admin

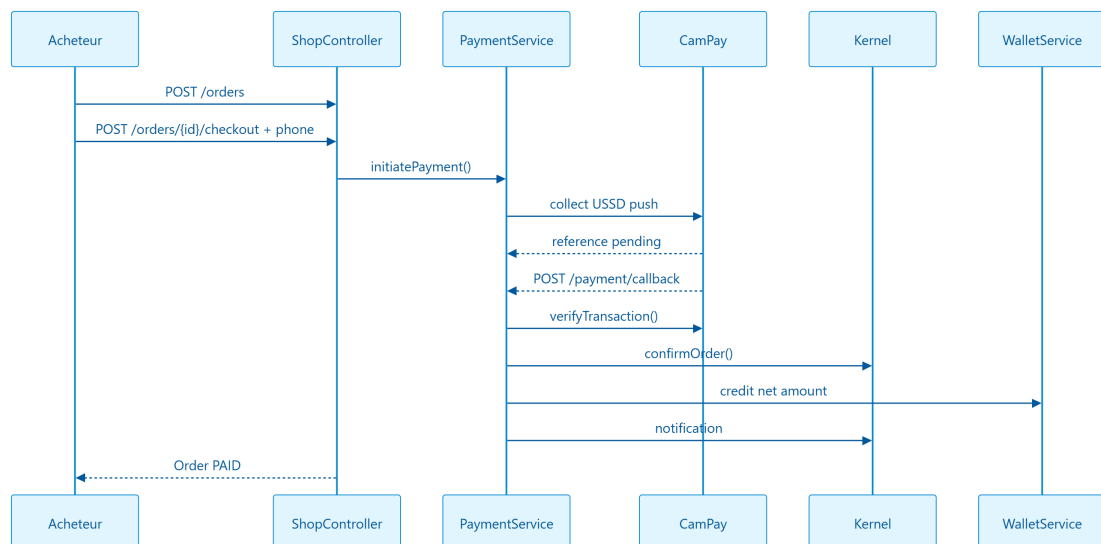


FIGURE 8.3 – Séquence — Paiement commande boutique via CamPay

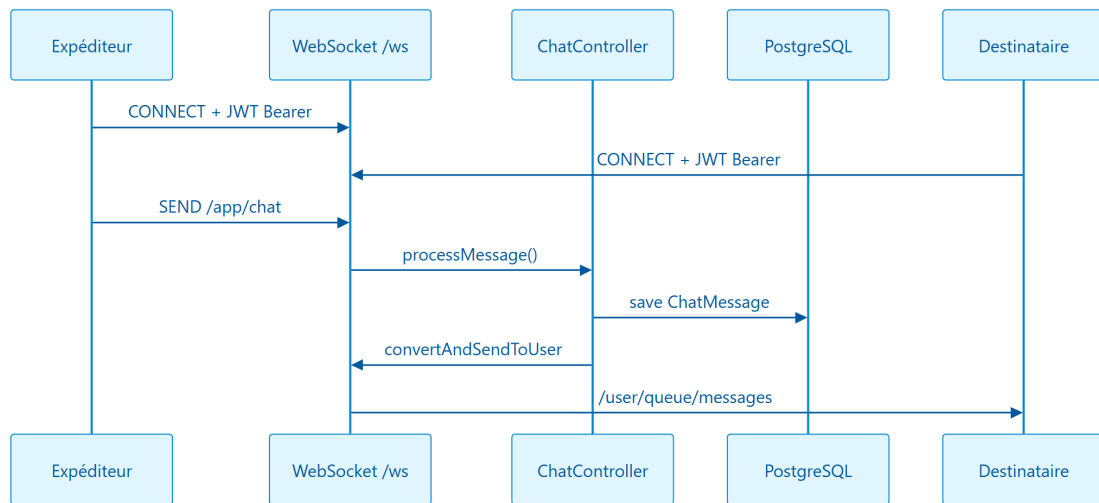


FIGURE 8.4 – Séquence — Envoi message chat temps réel

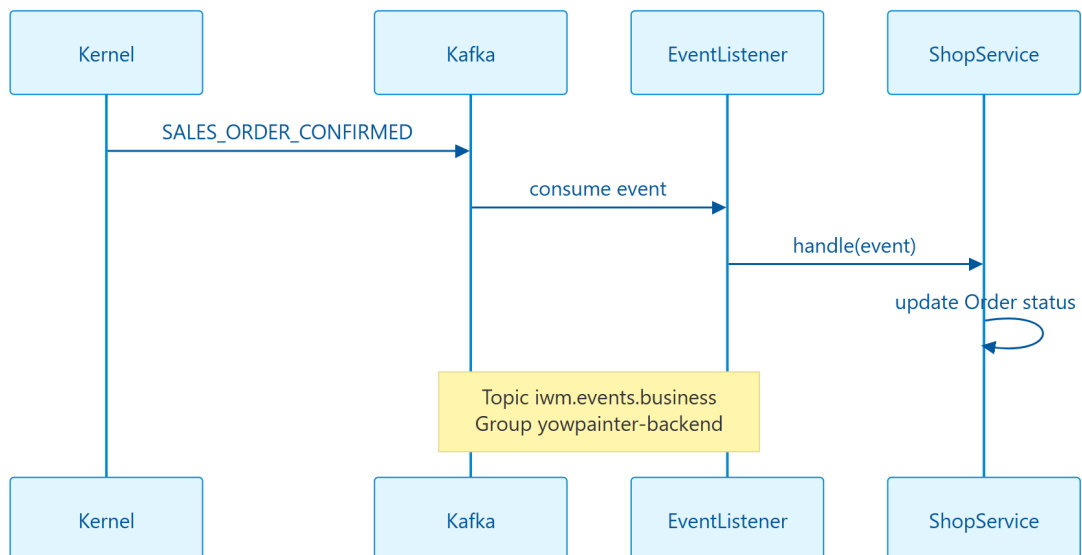


FIGURE 8.5 – Séquence — Synchronisation statut commande via Kafka

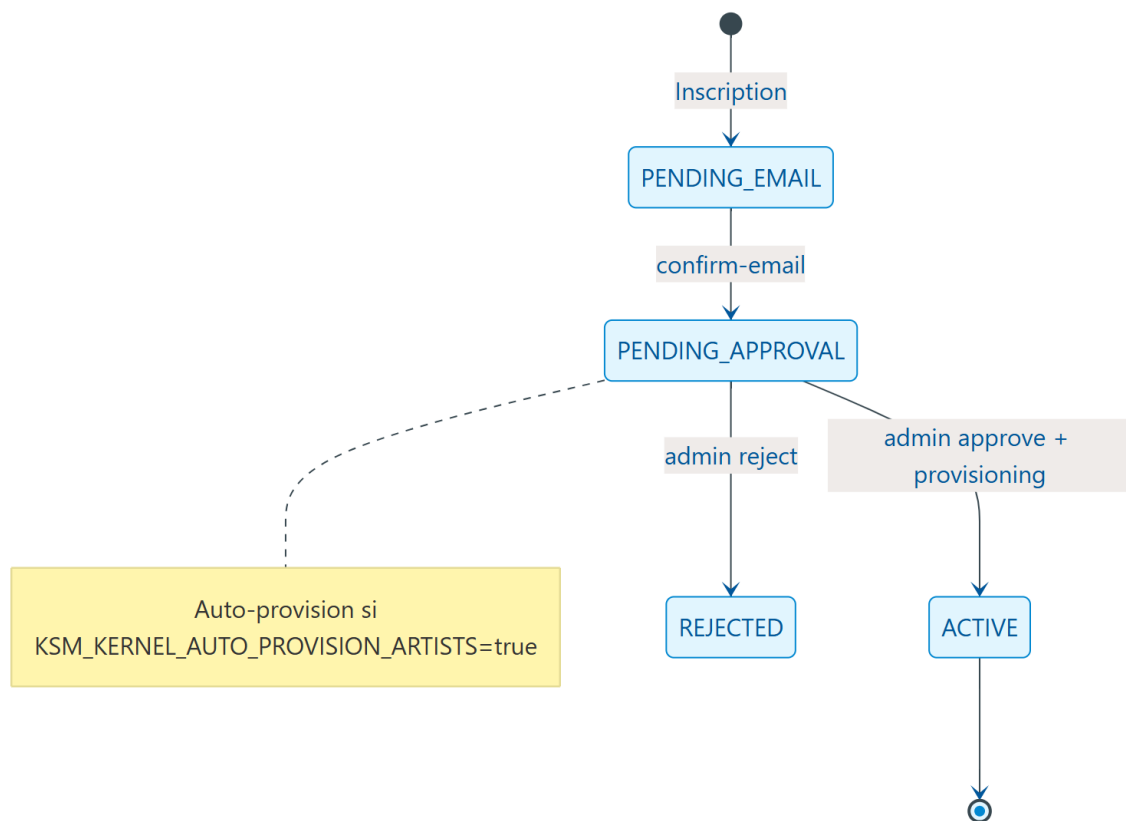


FIGURE 8.6 – Machine d'états — Statut artiste

Chapitre 9

Déploiement et infrastructure

9.1 Diagramme de déploiement — Environnement local

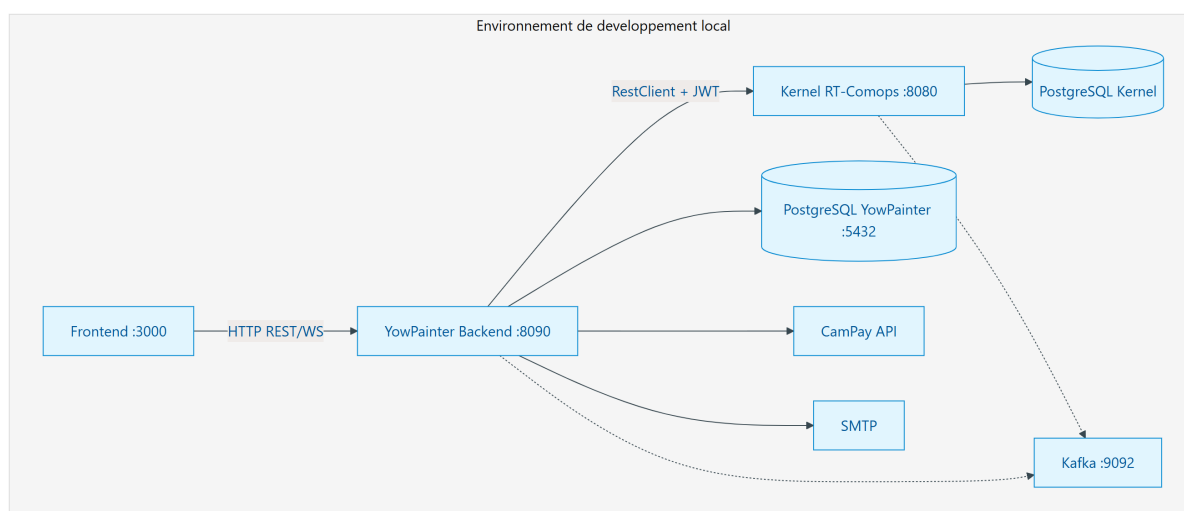


FIGURE 9.1 – Diagramme de déploiement — Développement local

9.2 Diagramme de déploiement — Production (cible)

9.3 Variables d'environnement critiques

TABLE 9.1: Variables d'environnement principales

Variable	Exemple	Description
PORT	8090	Port HTTP du backend
SPRING_DATASOURCE_URL	jdbc :postgresql ://...	Connexion PostgreSQL
KSM_KERNEL_BASE_URL	http ://localhost :8080	URL du Kernel
KSM_KERNEL_CLIENT_ID	yowpainter-backend	Identifiant ClientApplication
KSM_KERNEL_API_KEY	(secret)	Clé API Kernel
KSM_KERNEL_TENANT_ID	UUID	Tenant Kernel plateforme
KSM_KERNEL_JWK_SET_URI	/.well-known/jwks.json	Validation JWT RS256
KSM_KERNEL_AUTO_PROVISION_ARTISTS	false	Provisioning auto vs manuel

Variable	Exemple	Description
KSM_KERNEL_KAFKA_ENABLED	true/false	Activation consumer Kafka
APP_PAYMENT_CAMPAY_*	...	Credentials CamPay
APP_FRONTEND_URL	http ://localhost :3000	URL frontend (emails, redirects)

9.4 Pipeline de build et déploiement

1. **Build** : `mvn clean install` (compilation, tests unitaires)
2. **Tests** : `mvn test` (H2 en mémoire pour tests)
3. **Package** : JAR exécutable Spring Boot (`target/yowpainter-backend-*.jar`)
4. **Migrations** : Flyway exécuté au démarrage (schéma public + tenants actifs)
5. **Démarrage** : `java -jar app.jar` ou `.\scripts\run-local.ps1`
6. **Vérification** : `GET /api/public/health` + Swagger UI

9.5 Monitoring et observabilité

TABLE 9.2 – Points de contrôle disponibles

Endpoint	Auth	Usage
GET /api/public/health	Public	Health check externe (load balancer)
GET /api/system/health	JWT	Health détaillé (interne)
GET /api/system/version	JWT	Version application
GET /api/system/metrics	JWT	Métriques basiques
GET /api/admin/stats	ADMIN	Statistiques plateforme
GET /api/admin/logs	ADMIN	Logs récents

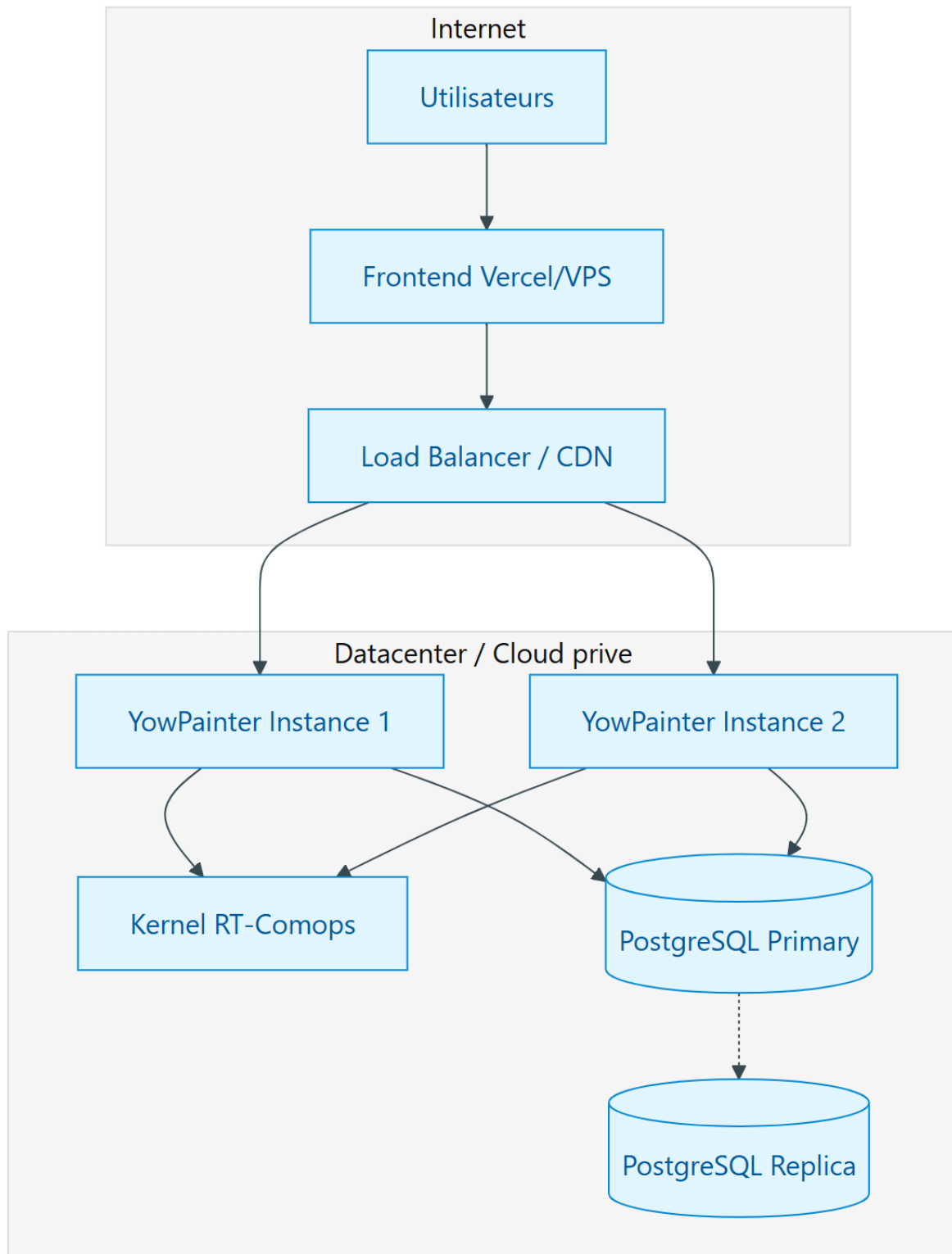


FIGURE 9.2 – Diagramme de déploiement — Architecture production cible

Chapitre 10

Référence API REST

10.1 Conventions générales

- **Base URL** : `http://localhost:8090` (dev) ou URL de production
- **Format** : JSON (Content-Type: `application/json`)
- **Authentification** : `Authorization: Bearer <JWT>` (sauf routes publiques)
- **Documentation interactive** : `/swagger-ui.html`
- **Multi-tenant** : header optionnel `X-Organization-Id: <UUID>`

10.2 Matrice des endpoints par module

10.2.1 Authentification — `/api/auth`

TABLE 10.1: Endpoints Auth

Méthode	Path	Auth	Description
GET	<code>/roles</code>	Public	Liste des rôles disponibles
POST	<code>/register</code>	Public	Inscription artiste/acheteur
POST	<code>/register/avatar</code>	Public	Upload avatar inscription
POST	<code>/login</code>	Public	Connexion (proxy Kernel)
POST	<code>/refresh</code>	Public	Renouvellement token
POST	<code>/logout</code>	JWT	Déconnexion
POST	<code>/forgot-password</code>	Public	Demande reset mot de passe
POST	<code>/reset-password</code>	Public	Réinitialisation mot de passe
POST	<code>/confirm-email</code>	Public	Confirmation e-mail
GET	<code>/confirm-email</code>	Public	Redirect confirmation

10.2.2 Profil — `/api/me`

POST	<code>/profile-picture</code>	JWT	Upload photo de profil (Kernel)
POST	<code>/files/upload</code>	JWT	Upload fichier générique

10.2.3 Artiste — `/api`

TABLE 10.2: Endpoints Artiste

Méthode	Path	Auth	Description
GET	/public/artists/{slug}	Public	Profil public artiste
GET	/public/artists/search	Public	Recherche artistes
GET	/artist/me	ARTIST	Mon profil artiste
PUT	/artist/me	ARTIST	Modifier profil
GET	/artist/me/analytics	ARTIST	Statistiques

10.2.4 Œuvres — /api

TABLE 10.3: Endpoints Artwork

Méthode	Path	Auth	Description
GET	/v1/public/{slug}/artworks	Public	Liste œuvres
GET	/v1/public/{slug}/artworks/{id}	Public	Détail œuvre
POST	/artworks	ARTIST	Créer œuvre
PUT	/artworks/{id}	ARTIST	Modifier
PATCH	/artworks/{id}/status	ARTIST	Changer statut
POST	/artworks/images/upload	ARTIST	Upload image
POST	/v1/public/{slug}/artworks/{id}/like	ARTIST/BUYER	Liker
POST	/v1/public/{slug}/artworks/{id}/comments	ARTIST/BUYER	Commenter
GET	/public/artworks/latest	Public	Dernières œuvres
GET	/public/artworks/featured	Public	Œuvres mises en avant

10.2.5 Boutique — /api/shop

TABLE 10.4: Endpoints Shop

Méthode	Path	Auth	Description
POST	/products	ARTIST	Créer produit
GET	/v1/public/{slug}/products	Public	Catalogue artiste
POST	/v1/public/{slug}/orders	BUYER	Créer commande
POST	/orders/{id}/checkout	BUYER	Payer (CamPay)
GET	/orders/my-purchases	BUYER	Mes achats
GET	/orders/my-sales	ARTIST	Mes ventes
PATCH	/orders/{id}/status	ARTIST	Mettre à jour statut
GET	/inventory	ARTIST	Stock produits

10.2.6 Événements — /api

TABLE 10.5: Endpoints Event

Méthode	Path	Auth	Description
GET	/public/events	Public	Liste événements
GET	/v1/public/{slug}/events	Public	Événements artiste

POST	/events	ARTIST	Créer événement
POST	/events/{eventId}/reservations	BUYER/ARTIST	Réserver
POST	/events/reservations/{id}/checkout	BUYER/ARTIST	Payer billet
POST	/events/tickets/validate	ARTIST	Valider QR code

10.2.7 Paiement et Wallet

TABLE 10.6 – Endpoints Payment / Wallet

Méthode	Path	Auth	Description
POST	/api/payment/callback	Public (webhook)	Callback CamPay
GET	/api/payment/history	JWT	Historique paiements
GET	/api/wallet/balance	ARTIST	Solde wallet
GET	/api/wallet/transactions	ARTIST	Transactions
POST	/api/wallet/withdraw	ARTIST	Retrait Mobile Money
POST	/api/wallet/settings/payout	ARTIST	Config retrait

10.2.8 Administration — /api/admin

TABLE 10.7: Endpoints Admin

Méthode	Path	Auth	Description
GET	/artists/pending	ADMIN	File d'attente artistes
POST	/artists/{id}/approve	ADMIN	Approuver + provisionner
POST	/artists/{id}/reject	ADMIN	Refuser inscription
GET	/tenants	ADMIN	Liste tenants
PATCH	/tenants/{id}/status	ADMIN	Activer/désactiver tenant
GET	/users	ADMIN	Liste utilisateurs
DELETE	/users/{id}	ADMIN	Supprimer utilisateur
GET	/stats	ADMIN	Statistiques plateforme
GET	/logs	ADMIN	Logs récents

10.3 WebSocket STOMP

TABLE 10.8 – Configuration WebSocket

Paramètre	Valeur
Endpoint connexion	ws://host:8090/ws (SockJS supporté)
Authentification	Header Authorization: Bearer <JWT>
Destination envoi	/app/chat
Destination réception	/user/queue/messages
Payload JSON	{senderId, recipientId, content, timestamp}

10.4 Codes de réponse HTTP

TABLE 10.9 – Codes HTTP standards utilisés

Code	Signification
200	Succès
201	Ressource créée
400	Requête invalide / validation échouée
401	Non authentifié / JWT invalide
403	Accès interdit (rôle insuffisant)
404	Ressource introuvable
409	Conflit (ex : e-mail déjà utilisé)
500	Erreur serveur (Kernel indisponible, etc.)

Chapitre 11

Annexes

11.1 Glossaire

TABLE 11.1: Glossaire des termes

Terme	Définition
Tenant	Espace de données isolé correspondant à un artiste (schéma PostgreSQL dédié)
Slug	Identifiant URL-friendly unique de l'artiste (ex : <code>lanslo-47d14</code>)
Kernel	Plateforme RT-Comops fournissant IAM, commerce, fichiers et notifications
ClientApplication	Application enregistrée dans le Kernel avec client-id et api-key
Provisioning	Création automatique de l'organisation Kernel et du schéma tenant
CamPay	Passerelle de paiement Mobile Money (MTN, Orange, MoMo Pay) au Cameroun
USSD Push	Notification de paiement envoyée sur le téléphone de l'acheteur
Port (hexagonal)	Interface définissant un contrat entre le domaine et l'infrastructure
Adaptateur	Implémentation concrète d'un port (JPA, HTTP, etc.)
JWKS	JSON Web Key Set — clés publiques pour validation JWT RS256
STOMP	Protocole de messagerie sur WebSocket
Flyway	Outil de migration de schéma de base de données versionné

TABLE 11.2 – Traçabilité BF → Modules

Besoin	Module(s)
BF-01 à BF-03	auth
BF-04, BF-17	admin, auth
BF-05	artist
BF-06	artwork, file
BF-07	shop, payment
BF-08	event, payment
BF-09, BF-10	payment
BF-11	subscription, payment
BF-12	chat
BF-13	notification
BF-14	news
BF-15	search
BF-16	file

TABLE 11.3 – Tables du schéma public

Table	Contenu
app_user	Utilisateurs (artistes, acheteurs, admins)
artist	Extension profil artiste (JOINED inheritance)
wallet	Portefeuille artiste
wallet_transaction	Mouvements wallet
chat_message	Messages chat
chat_room	Salles de discussion
refresh_token	Tokens de rafraîchissement
pending_provision_session	Sessions MFA provisioning

11.2 Matrice de traçabilité besoins / modules

11.3 Structure des schémas de base de données

11.3.1 Schéma public

11.3.2 Schéma tenant_{organizationId}

11.4 Checklist de mise en production

1. Créer `ClientApplication yowpainter-backend` dans le Kernel
2. Configurer toutes les variables d'environnement (pas de secrets en dur)
3. Définir `KSM_KERNEL_AUTO_PROVISION_ARTISTS=false` pour validation manuelle
4. Configurer credentials bootstrap admin + MFA
5. Vérifier connexion PostgreSQL et droits création schémas
6. Configurer CamPay production (callback URL publique)
7. Configurer SMTP pour e-mails transactionnels
8. Activer Kafka si synchronisation commandes requise
9. Sécuriser routes chat (`permitAll` → authentification requise)
10. Tester health checks et Swagger
11. Exécuter `mvn test` en CI

TABLE 11.4 – Tables par tenant artiste

Table	Contenu
artwork, artwork_image, artwork_video	Œuvres et médias
artwork_like, artwork_comment	Interactions sociales
product, order, order_item	Commerce
payment	Transactions CamPay
event, reservation, ticket	Événements et billets
news	Actualités
subscription	Abonnement artiste

11.5 Historique des versions du document

TABLE 11.5 – Historique documentaire

Version	Date	Auteur	Modifications
1.0	Juillet 2026	Équipe YowPainter	Création initiale du CAC

11.6 Index des figures

Ce document contient les diagrammes suivants :

- Figure 2.1 — Diagramme de contexte
- Figure 2.2 — Isolation multi-tenant
- Figure 4.1 — Cas d'utilisation globaux
- Figure 5.1 — Architecture hexagonale
- Figure 5.2 — Diagramme de packages
- Figure 5.3 — Diagramme de composants
- Figures 6.1 à 6.4 — Classes métier
- Figures 7.1 à 7.5 — Classes techniques
- Figures 8.1 à 8.5 — Diagrammes de séquence
- Figure 8.6 — Machine d'états artiste
- Figures 9.1 et 9.2 — Déploiement